Code, Community & AI

Ποιος θα συντηρεί τον κώδικα όταν τελειώσει η μαγεία;



GDG Cloud Thessaloniki

780 Group members

Joined

About | Upcoming events | Past events | Organizers | Photos | Video

About

https://gdg.community.dev/gdg-cloud-thessaloniki/

Thessaloniki is rapidly evolving into a major innovation hub where deep academic heritage meets a rising tech ecosystem and **GDG Cloud Thessaloniki** sits right at the heart of this momentum.

We built an open, hands-on, and forward-looking space where modern cloud architecture, artificial intelligence, and open engineering become accessible to everyone. Here, technology isn't abstract theory, it is practical, empowering, and community-driven.

Our community brings together university students, DevOps specialists, cloud architects, researchers, creative builders, and tech professionals under one collaborative umbrella. Through technical workshops, deep-dive sessions, hackathons, and open dialogues, curiosity transforms into production-ready skills, and casual meetups spark lasting partnerships.

Our mission is clear: we demystify cutting-edge cloud infrastructure and emerging tech, bridge academia with modern industry demands, and champion knowledge sharing across Northern Greece and beyond.

Whether you are deploying mission-critical systems, exploring modern AI workflows, taking your first steps in the cloud, or switching careers into tech, you belong here. This is your platform to experiment, build, and grow.

✨ **Connect** with engineers, builders, and innovators across our vibrant tech ecosystem.

✨ **Learn by doing**, exchange architectural insights, and build with modern tools.

✨ **Help shape** Thessaloniki's digital future and regional impact.

Join our events, share your perspective, and build the future with us.



DevFest 2026 - Thessaloniki, Greece

OK! Thess, 2, Komotinis St, Thessaloniki, 54655

GDG Cloud Thessaloniki

Join GDG Cloud Thessaloniki DevFest 2026, the ultimate event for tech enthusiasts. Dive into keynotes and sessions on cloud-native patterns, AI, and web architectures. Network with like-minded pros, enjoy top-tier swag, and connect with future collaborators in the heart of Thessaloniki. No buzzwords, just real tech insights.

Your ticket gives you access to in-person event venues.

https://gdg.community.dev/events/details/google-gdg-cloud-thessaloniki-presents-devfest-2026-thessaloniki-greece/

Dec 12, 9:00 AM – 6:00 PM (GMT+2) 31 RSVP'd

Your RSVP

Key themes

Conference DevFest

About this event

Calling all infrastructure ninjas, terminal dwellers, code craftspeople, and production heroes who never push directly to main on a Friday! 🚀

Clear your cache and mark your calendars: **GDG Cloud Thessaloniki DevFest 2026** is landing right here in the heart of Thessaloniki—and it's our biggest celebration of modern engineering yet! 🇬🇷

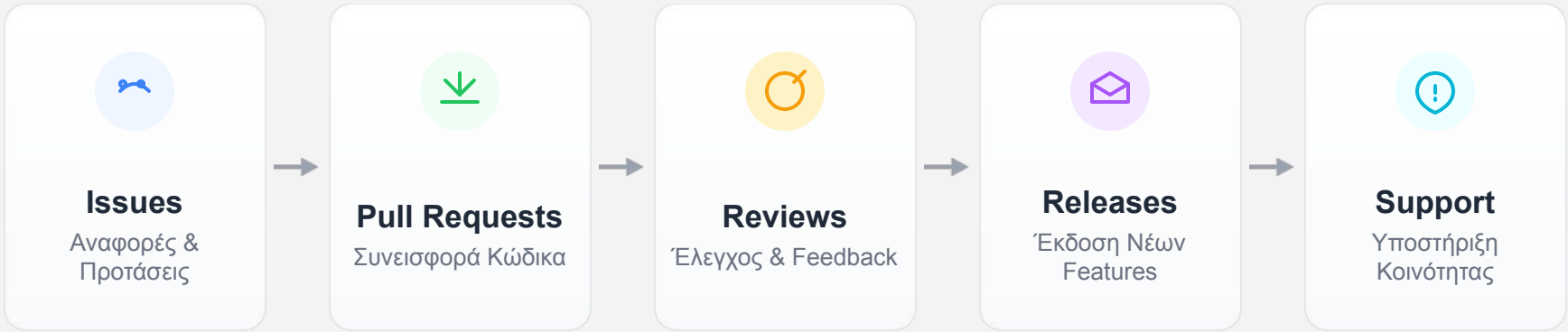
DevFest 2026
12 Δεκεμβρίου
OK!Thess

Το ελεύθερο λογισμικό δεν γράφεται μόνο του

- Κώδικας
- Τεκμηρίωση
- Μετάφραση
- Κοινότητα (άνθρωποι)

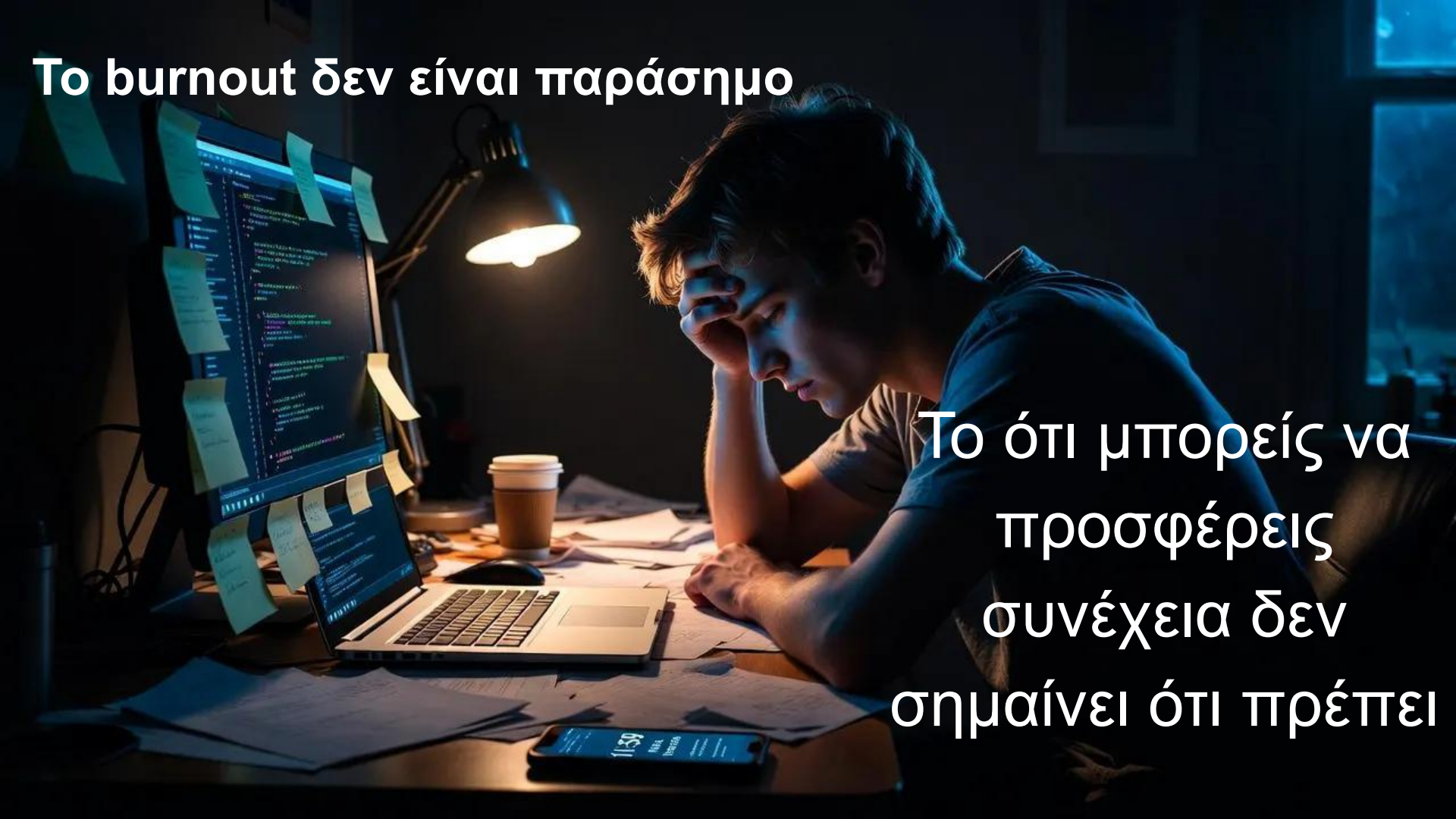


Πίσω από κάθε project υπάρχει ένας άνθρωπος



Κάθε στάδιο του κύκλου ανάπτυξης βασίζεται στην προσπάθεια και τη συνεργασία των ανθρώπων της κοινότητας.

Το burnout δεν είναι παράσημο



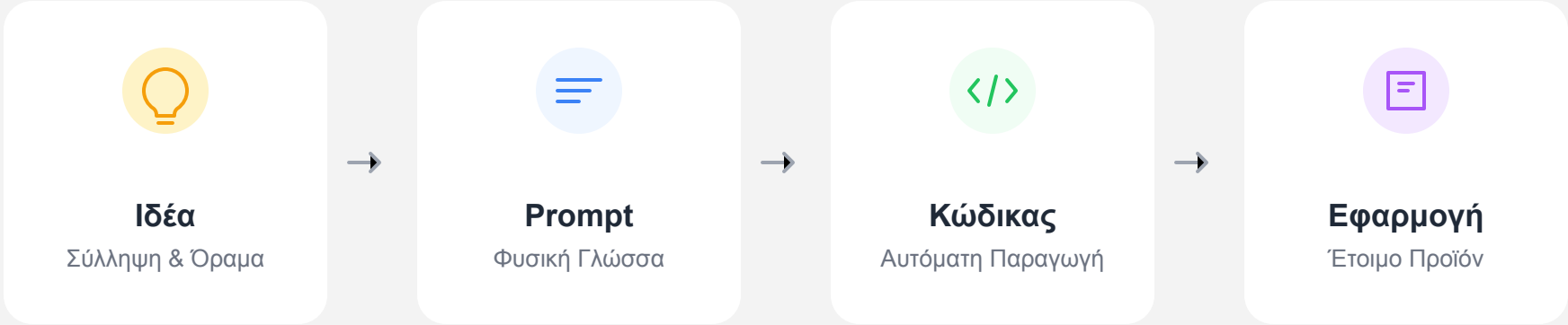
Το ότι μπορείς να προσφέρεις συνέχεια δεν σημαίνει ότι πρέπει

Κάνε διάλειμμα. Το project θα περιμένει.



- Παύση
- Όρια
- Βιωσιμότητα

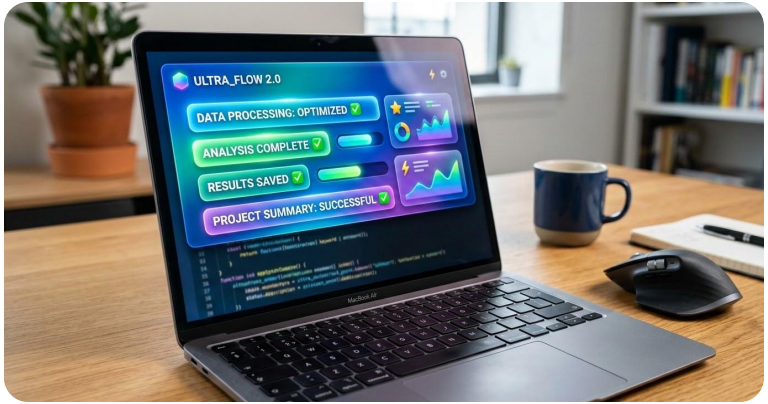
Από την ιδέα στην εφαρμογή με φυσική γλώσσα



Μετασχηματίζοντας τις σκέψεις σε λειτουργικό λογισμικό άμεσα και απλά.

ΠΑΓΙΔΑ: δεν σημαίνει ότι το λογισμικό είναι ασφαλές ή έτοιμο για παραγωγή

Το demo λειτουργεί. Μήπως και η εφαρμογή;



Φαίνεται ότι λειτουργεί

Επιφανειακή επίδειξη (Demo) χωρίς εξαντλητικούς ελέγχους, δοκιμασμένη μόνο σε ιδανικές συνθήκες (happy path).



Έχει επαληθευτεί ότι λειτουργεί

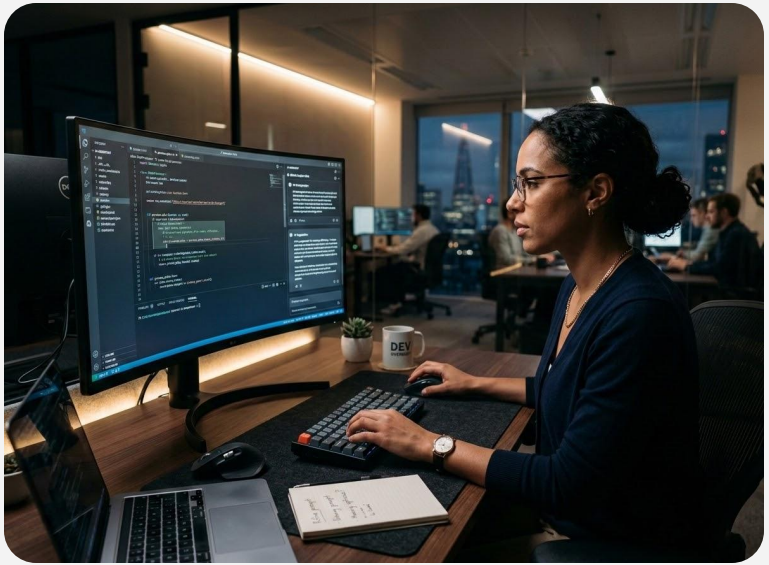
Πλήρως ελεγμένο λογισμικό με αυτοματοποιημένα tests, ασφάλεια, αντοχή σε σφάλματα και έτοιμο για παραγωγή.

Η ΤΝ γράφει. Εσύ παραμένεις υπεύθυνος

ΤΝ = επιταχυντής

όχι υποκατάστατο κρίσης

Η τεχνητή νοημοσύνη προσφέρει ταχύτητα και παραγωγικότητα, αλλά η τελική ευθύνη, η επαλήθευση και η λήψη αποφάσεων ανήκουν αποκλειστικά στον άνθρωπο.



Μην ξεκινάς με κώδικα. Ξεκίνα με σχέδιο.

1. Requirements → **2. Plan (PRD)** → 3. Review → 4. Implement

Η στρατηγική του Plan Mode

Η πρώτη παγίδα είναι να ζητήσουμε αμέσως "φτιάξε μου μια εφαρμογή". Πριν τον κώδικα, ορίζουμε τι θέλουμε, ποιος θα το χρησιμοποιεί & τι συνιστά επιτυχία.

Προτείνεται ένα **μικρό PRD σε Markdown**. Ζητάμε από την AI να παρουσιάσει το σχέδιο, τα αρχεία που θα αλλάξει και τις υποθέσεις της για διόρθωση πριν την υλοποίηση.

«Είναι πολύ φθηνότερο να διορθώσεις μια λάθος ιδέα σε ένα Markdown αρχείο παρά να ξηλώσεις μια εφαρμογή.»

Πρακτικό παράδειγμα

Εφαρμογή καταγραφής εργασιών (Task Tracker)

Πριν γραφτεί οτιδήποτε, ορίζουμε στο PRD:

- **Ρόλοι Χρηστών:** Διαχειριστές, Μέλη ομάδας, Επισκέπτες
- **Βασικές Λειτουργίες:** Δημιουργία, ανάθεση & φιλτράρισμα tasks
- **Αποθήκευση Δεδομένων:** Δομή βάσης & μοντέλα
- **Κριτήρια Αποδοχής:** Ορισμός επιτυχίας & edge cases

Το καλό prompt μοιάζει με καλό τεχνικό ticket

ΠΡΕΠΕΙ να αντιμετωπίζουμε τα prompts όπως τα τεχνικά tickets μιας ομάδας ανάπτυξης. Όσο πιο ξεκάθαρος είναι ο ορισμός του προβλήματος, τόσο λιγότερο αφήνουμε το μοντέλο να μαντεύει.

1. Context

Πρώτα εξηγούμε το πλαίσιο της εφαρμογής και τον στόχο.

2. To-do

Περιγράφουμε αναλυτικά τι ακριβώς πρέπει να γίνει.

3. Not-to-do

Διευκρινίζουμε τι να αποφευχθεί ή περιορισμούς.

4. Acceptance Criteria

Ορίζουμε τα κριτήρια επιτυχίας & tests.

✕ Ασαφές Prompt (Bad Practice)

«Φτιάξε μου ένα login.»

Προβλήματα:

- Δεν υπάρχει πλαίσιο τεχνολογιών.
- Ασαφής χειρισμός σφαλμάτων.
- Κανένας ορισμός ασφάλειας.
- Η ΤΝ υποθέτει τα πάντα μόνη της, οδηγώντας σε μη παραγωγικό κώδικα.

✓ Structured Prompt Ticket (Good Practice)

Context: Node.js/Express API με PostgreSQL.
To-do: Δημιούργησε endpoint `/login` με email & password. Επίστρεψε JWT token σε επιτυχία.
Not-to-do: Μην αποθηκεύσεις plaintext passwords, μην χρησιμοποιήσεις session cookies.
Acceptance Criteria:
1. Σαφής χειρισμός αποτυχημένης σύνδεσης (401 Unauthorized).
2. Γράψε unit tests για επιτυχημένα & αποτυχημένα σενάρια.

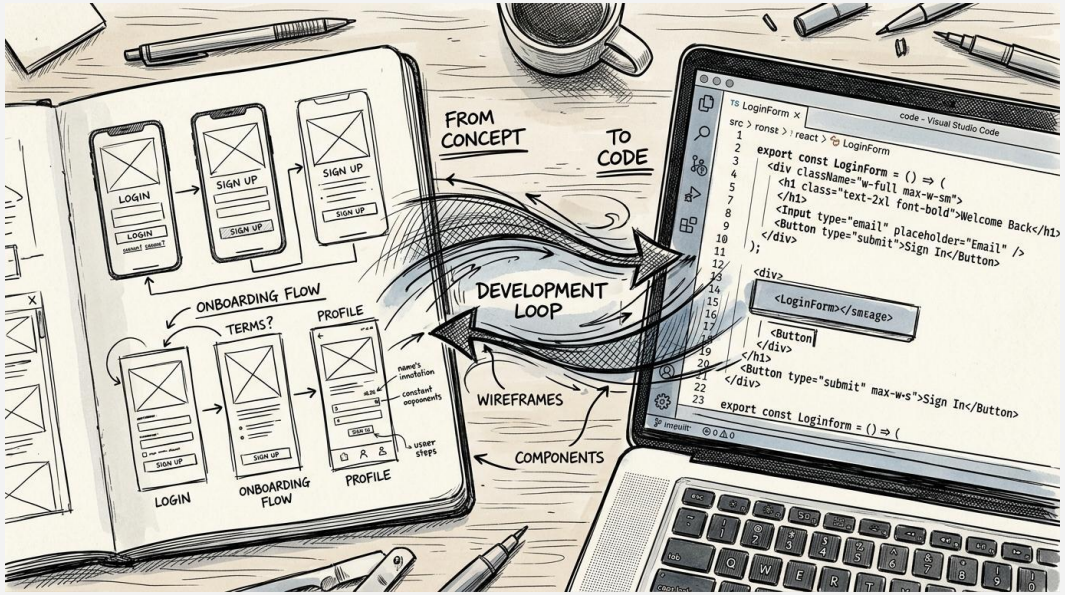
Δώσε στην AI το σωστό context

1. Κώδικας
Δομή έργου & υφιστάμενες υλοποιήσεις

2. Τεκμηρίωση
Docs βιβλιοθηκών & APIs

3. Visual Context
Screenshots & σχεδιαγράμματα UI

4. Κανόνες Έργου
Συμβάσεις & περιορισμοί (AGENTS.md)



«Το context είναι μέρος της μηχανικής, όχι απλώς μέρος του prompt.»

Το Git είναι το δίχτυ ασφαλείας σου

1. Checkpoint
Καθαρή αρχική κατάσταση

→

2. Αλλαγή
Μικρά, εστιασμένα edits

→

3. Έλεγχος
Επιθεώρηση diff & tests

→

4. Commit / Rollback
Αποδοχή ή επαναφορά

Έλεγχος αλλαγών με Git

Χωρίς checkpoints, μια μικρή αλλαγή από την AI μπορεί να καταστρέψει κάτι που λειτουργούσε.

Τι εξετάζουμε στο Diff:

- **Αλλαγές:** Τι προστέθηκε & τι διαγράφηκε
- **Εξαρτήσεις:** Νέες ή τροποποιημένες βιβλιοθήκες
- **Επίπτωση:** Αν επηρεάζονται κρίσιμες λειτουργίες

⚠️ Βασικός κανόνας

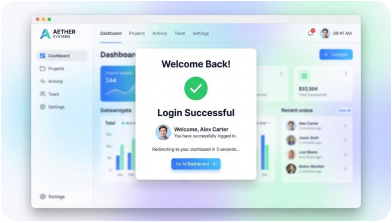
Μην αποδέχεσαι ένα μεγάλο σύνολο αλλαγών μόνο και μόνο επειδή η TN αναφέρει ότι ολοκλήρωσε επιτυχώς την εργασία.

• Χρησιμοποιούμε **ξεχωριστά branches** για πειραματισμούς.
• Κάνουμε **συχνά, μικρά commits** σε κάθε επιτυχημένο βήμα.
• Το Git μας επιτρέπει να επιστρέφουμε πάντα σε μια **γνωστή καλή κατάσταση**.

Test πρώτα, εμπιστοσύνη μετά

Η ΤΝ μπορεί να φτιάξει κώδικα που δουλεύει στο ιδανικό σενάριο, αλλά αποτυγχάνει σε πραγματικές συνθήκες. Η πλήρης κάλυψη απαιτεί έλεγχο όλων των πιθανών διαδρομών.

✓ Happy Path (Ιδανικό Σενάριο)



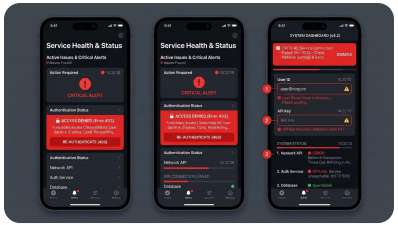
Επιτυχημένη ροή

- Σύνδεση έγκυρου χρήστη με σωστά διαπιστευτήρια.
- Ορθές εισαγωγές δεδομένων & αναμενόμενη συμπεριφορά.

Στόχος ελέγχου:

Επιβεβαίωση ότι η βασική λειτουργικότητα εκτελείται χωρίς σφάλματα υπό κανονικές συνθήκες.

✕ Unhappy Path (Οριακές Περιπτώσεις)



Δοκιμασία αντοχής

- Κενά πεδία & μη έγκυρα δεδομένα.
- Χρήστες χωρίς δικαιώματα & αποτυχία δικτύου.

Στόχος ελέγχου:

Διασφάλιση ότι το σύστημα διαχειρίζεται σωστά τα σφάλματα χωρίς να καταρρέει.

Βασικές πρακτικές επαλήθευσης:

- **Συνεχής έλεγχος:** Εκτέλεση σχετικών tests, type checking & linting μετά από κάθε μικρή αλλαγή.
- **Εντοπισμός Bugs:** Η επιτυχία των tests επιβεβαιώνει συγκεκριμένες συμπεριφορές, δεν αποδεικνύει την απουσία σφαλμάτων.

«Εξετάζουμε πάντα το αποτέλεσμα αντί να υποθέτουμε ότι όλα λειτουργούν ρολόι.»

Ασφάλεια: Μην εμπιστεύεσαι τυφλά τον κώδικα AI

Όταν διαχειριζόμαστε προσωπικά δεδομένα ή πληρωμές, το κόστος ενός λάθους είναι τεράστιο.

1. Authorization / Authentication
Έλεγχος πρόσβασης & ρόλων

2. Secrets
Διαχείριση κλειδιών

3. Validation
Επικύρωση εισόδων

4. Dependencies
Έλεγχος βιβλιοθηκών

5. Auditing
Στατικοί αναλυτές

Ανεξάρτητη επαλήθευση

- Δεν αρκούμαστε στις διαβεβαιώσεις της ίδιας της AI.
- Χρήση **SAST tools** & **dependency scanners**.
- Ο έλεγχος βασίζεται σε **πραγματικά ευρήματα**.

«Το Open Source προσφέρει διαφάνεια, αλλά δεν εγγυάται αυτόματα την ασφάλεια.»

⚠️ Κρίσιμη προειδοποίηση

ΠΟΤΕ μην κοινοποιείτε ή ανεβάζετε:

- API Keys & Passwords
- Serial Numbers & Διαπιστευτήρια
- Ιδιωτικά keys σε AI prompts ή Git repositories

Χρησιμοποιείτε πάντα περιβάλλον μεταβλητών (.env).

Βασική Αρχή Asymmetric Trust

Η παραγωγή κώδικα από την AI είναι γρήγορη, αλλά η ευθύνη της ασφάλειας παραμένει 100% στον μηχανικό. Κάθε γραμμή δικτύου, εισόδου και ταυτοποίησης πρέπει να αντιμετωπίζεται ως δυνητικά ευάλωτη.

Σπάσε τον φαύλο κύκλο του debugging

Διαχωρισμός διάγνωσης από τη διόρθωση για την αποφυγή του "Vibe Coding Doom Loop".

1. Observe
Ανάλυση logs & συμπτωμάτων

→

2. Diagnose
Εντοπισμός πιθανής αιτίας

→

3. Hypothesis
Εξήγηση στοιχείων

→

4. Fix
Ελεγχόμενη αλλαγή

→

5. Test
Επαλήθευση αποτελέσματος

⚠️ Vibe Coding Doom Loop

- Η ΤΝ διορθώνει ένα bug αλλά δημιουργεί νέο.
- Συνεχείς τυφλές διορθώσεις χωρίς διάγνωση.
- Άγνωστο ποια αλλαγή προκάλεσε ποιο πρόβλημα.

Μην στέλνεις απλώς το ίδιο prompt πιο δυνατά!

✓ Συστηματική προσέγγιση

1. Διάγνωση: Ζήτα ανάλυση logs χωρίς αλλαγή κώδικα.

2. Αξιολόγηση: Επιβεβαίωσε την πιθανή αιτία.

3. Διόρθωση: Εφάρμοσε μία ελεγχόμενη αλλαγή & test.

Αλλαγή στρατηγικής, καθαρότερο context ή human review.

Βασική Αρχή: Διαχωρισμός διάγνωσης & διόρθωσης

Πριν αλλάξει οτιδήποτε στον κώδικα, απαιτείται πλήρης κατανόηση του προβλήματος. Αν το μοντέλο επαναλαμβάνει λάθη, άλλαξε προσέγγιση ή δώσε καλύτερο context.

Μάθε πότε να σταματάς και να επανεκτιμάς

Η επιμονή είναι χρήσιμη, αλλά η ατέρμονη επανάληψη δεν είναι στρατηγική.



⚠️ Σημάδια ότι πρέπει να σταματήσεις

- **Επαναλαμβανόμενα bugs:** Η ΤΝ ξαναφέρνει το ίδιο πρόβλημα.
- **Cascading failures:** Μια διόρθωση σπάει μια άλλη λειτουργία.
- **Αυξανόμενη πολυπλοκότητα:** Ο κώδικας γίνεται πιο περίπλοκος χωρίς λόγο.

Μην συνεχίζεις το ίδιο μοτίβο ελπίζοντας σε διαφορετικό αποτέλεσμα!

✓ Εναλλακτικές στρατηγικές επανεκκίνησης

- **Νέα συνομιλία:** Ξεκίνα από την αρχή με καθαρή και ακριβή περιγραφή του προβλήματος.
- **Τεκμηρίωση & Review:** Συμβουλέψου την επίσημη τεκμηρίωση ή ζήτησε ανεξάρτητη αξιολόγηση.
- **Manual Implementation:** Γράψε εσύ το κρίσιμο τμήμα του κώδικα.

Βασική Αρχή: Εστίαση στο πραγματικό πρόβλημα

Το ζητούμενο δεν είναι να κερδίσεις μια διαφωνία με το μοντέλο. Είναι να λύσεις το πραγματικό πρόβλημα με τον λιγότερο κίνδυνο και τη μεγαλύτερη δυνατή σαφήνεια.

Ένα workflow που αξίζει να υιοθετήσεις

Μία ολοκληρωμένη ροή εργασίας για αποτελεσματική και ασφαλή ανάπτυξη με ΤΝ.

01
Σχεδίασε
Απαίτηση και κριτήρια αποδοχής

02
Συμφώνησε το πλάνο
Έλεγξε τις υποθέσεις της ΤΝ

03
Υλοποίησε μικρά βήματα
Ένα feature κάθε φορά

04
Έλεγξε τις αλλαγές
Diff, Git και code review

05
Επαλήθευσε
Tests, ασφάλεια και συμπεριφορά

06
Κάνε release και μάθε
Τεκμηρίωση και feedback



Βασική Αρχή: Συστηματική & Επαναλαμβανόμενη Διαδικασία

Η υιοθέτηση ενός δομημένου κύκλου εργασίας εξασφαλίζει υψηλή ποιότητα κώδικα, μειώνει τα σφάλματα και μεγιστοποιεί την παραγωγικότητα της ομάδας.


Ο στόχος δεν είναι να γράφουμε περισσότερο κώδικα
1. SPECIFY
(User Journeys & Experiences)
2. PLAN
(Technical Stack & Architecture)
3. TASKS
(Breakdown & Validation)
4. IMPLEMENT
(Execution & Focused Review)
Efstathios Iosifidis
https://www.linkedin.com/in/eiosifidis